PRODUCT REQUIREMENT DOCUMENT (PRD)

PRD-001: SERVERLESS OPEN-SOURCE LEETCODE-TO-GITHUB SYNC ENGINE

Authors: Principal Product Manager & Principal Software Architect

Target Platform: Google Chrome Extension (Manifest V3)

Distribution Model: Self-Hosted / Unpacked Developer Mode

1. EXECUTIVE SUMMARY & VALUE PROPOSITION

1.1 MISSION STATEMENT

To provide software engineers with an autonomous, enterprise-grade, serverless sync pipeline that archives successfully resolved LeetCode problems directly to a personal GitHub repository. The product eliminates reliance on third-party SaaS hosting, preserving user data privacy and absolute control over intellectual property without incurring infrastructure overhead.

1.2 CORE VALUE PROPOSITION

Current market solutions rely on centralized backend proxies to store OAuth tokens, introducing severe vector risks for supply-chain attacks and credential theft. This extension delivers a **Zero-Server Security Architecture**, running entirely inside the client's browser runtime. It guarantees zero data collection, zero operating costs, and zero external dependencies, establishing a direct, cryptographically secured bridge between LeetCode and GitHub.

1.3 TARGET AUDIENCE SEGMENTATIONS

Audience Segment	Primary Pain Points	Core Value Delivered
DSA Students & Job Seekers	High friction in manually documenting solutions; lack of a	Automated portfolio generation; instantaneous, structured

	structured, visible technical portfolio for recruiters.	code commits showing daily algorithmic consistency.
Open-Source Maintainers	Reluctance to authorize third-party OAuth access to their primary GitHub profiles due to security anxieties.	Pure source-code auditability; local compilation; complete assurance that credentials never leave local storage.
Active Competitive Programmers	Time lost switching between the browser IDE and terminal environments to	Transparent background execution; immediate repository synchronization

	maintain a clean Git tree.	upon problem acceptance.

2. ULTRA-SECURE CLIENT-SIDE ARCHITECTURE (ZERO-SERVER SECURITY)

Because this extension is distributed exclusively as uncompiled open-source code loaded via Chrome's "Load unpacked" mode, it cannot utilize an hidden client secret. The entire execution architecture must be hardened against memory extraction, local malicious script injection, and cross-site scripting (XSS).

2.1 CRYPTOGRAPHIC STORAGE OF GITHUB PERSONAL ACCESS TOKEN (PAT)

The extension mandates a scoped GitHub Personal Access Token (PAT) provided by the user. To prevent physical/malware extraction from the plaintext SQLite files of `chrome.storage.local`, all credentials must be encrypted using 256-bit AES-GCM via the Web Crypto API.

Key Derivation Protocol (PBKDF2)

1. **Master Secret:** The user creates a custom Master PIN/Passphrase during onboarding. This passphrase is never written to disk.
2. **Salt Generation:** On initial setup, a cryptographically secure pseudo-random 16-byte salt is generated via `crypto.getRandomValues()`. This salt is stored unencrypted in `chrome.storage.local` to facilitate future key derivations.
3. **Derivation:** The passphrase and salt are processed through PBKDF2 (`HMAC-SHA-256`, 100,000 iterations) to output a 256-bit symmetric encryption key.

Encryption & Storage Sequence

1. The derived key initializes the AES-GCM algorithm.
2. A unique 12-byte Initialization Vector (IV) is generated for each write operation.
3. The raw GitHub PAT is encrypted, outputting an `ArrayBuffer` of ciphertext and an authentication tag (inherent to GCM).
4. The payload is written to `chrome.storage.local` utilizing the following schema:

JSON

```
{
  "crypto_metadata": {
    "salt": "base64_encoded_16_byte_salt",
    "iv": "base64_encoded_12_byte_iv"
  },
  "secure_payload": "base64_encoded_ciphertext_plus_tag"
}
```

Ephemeral Session Lifetime

To preserve a frictionless user experience while adhering to zero-server paradigms, the derived encryption key or decrypted token must only be cached within `chrome.storage.session` (in-memory, cleared completely upon closing the browser session). It must never persist in long-term storage.

2.2 SECURE INTER-SCRIPT MESSAGE PASSING

Manifest V3 isolates components into distinct runtime worlds. Communication between the Main World (LeetCode page context), the Isolated World (Content Script), and the Extension Background Service Worker must be strictly validated to prevent arbitrary script injections.

Main-World-to-Isolated-World Bridge

1. During initialization, the Content Script generates a cryptographically random 32-character alpha-numeric session token (`MessageNonce`).
2. The Content Script injects the Main World script (the hook) into the DOM, explicitly passing the `MessageNonce` via a single-use custom DOM event.
3. When the Main World script intercepts a successful submission, it broadcasts the payload via `window.postMessage`. The payload object must strictly contain the correct `MessageNonce`.
4. The Content Script catches the `message` event, validates that `event.origin` matches `https://leetcode.com`, and verifies that `event.data.nonce === MessageNonce`. If it does not match, the message is discarded.

Isolated-World-to-Background Service Worker Channel

1. Communication uses `chrome.runtime.sendMessage`.
2. The Background Service Worker does not declare an `externally_connectable` match pattern in `manifest.json`, automatically dropping all external message attempts originating outside the extension's execution context.
3. Every internal message payload must adhere to a strict internal interface. The worker validates the presence of an expected structural schema before acting on the message:

TypeScript

```
interface InternalMessage {
  source: "LEETCODE_CONTENT_SCRIPT";
  action: "PROCESS_SUBMISSION";
  validationHash: string; // SHA-256 hash of (Problem_ID + Submission_ID)
  payload: object;
}
```

3. LEETCODE SUBMISSION DETECTION & AUTOMATED SCRAPING

Data collection must combine low-level API hooking with high-level DOM tracking to ensure maximum resilience against frequent LeetCode UI updates.

3.1 DUAL-LAYER HOOKING & MUTATION STRATEGY

Because LeetCode executes as a Single Page Application (SPA), relying exclusively on DOM queries will cause synchronization failures during high-latency UI renders.

Layer 1: Main World Network Interception (Primary)

The extension injects a script into the page context (`world: "MAIN"`) at `document_start`. This script overrides the global `window.fetch` and `XMLHttpRequest` prototypes.

- **Target Endpoints:**
  - *Old/Classic UI:* Intercept HTTP POST requests heading to `https://leetcode.com/submissions/detail/*/check/`.
  - *New/GraphQL UI:* Intercept HTTP POST requests to `https://leetcode.com/graphql/` matching the operation name `submitSubmission` or `submissionDetails`.
- **Interception Logic:** The hook monitors the network response stream. It parses the incoming JSON text. If the payload indicates an execution state of `status_msg === "Accepted"` (or state code corresponding to successful validation), it instantly extracts the raw telemetry (e.g., `submission_id`, memory, runtime) and forwards it to Layer 2.

Layer 2: MutationObserver UI Binding (Secondary Validation)

Simultaneously, the Content Script runs an active `MutationObserver` targeting the core application layout wrapper (`#id` or class roots varying across layouts).

- **Old UI Target:** Observe container `.submission-status-container`.
- **New UI Target:** Observe container `div[data-e2e-locator="submission-result"]`.
- **Condition Trigger:** The observer triggers when the text content transitions into a green success state matching the exact string `"Accepted"`. This

validation unblocks the internal pipeline, linking the user interface update with the intercepted network payload from Layer 1.

3.2 STANDARDIZED EXTRACTION DATA PAYLOAD

Once both layers validate a successful submission, the extension builds the following structured JSON schema:

JSON

```
{
  "metadata": {
    "sync_engine_version": "1.0.0",
    "timestamp_utc": "2026-06-22T18:20:56Z"
  },
  "problem": {
    "id": "1",
    "slug": "two-sum",
    "title": "Two Sum",
    "difficulty": "Easy",
    "url": "https://leetcode.com/problems/two-sum/"
  },
  "submission": {
    "id": "987654321",
    "language": "python3",
    "language_extension": "py",
    "runtime_ms": 32,
    "runtime_percentile": 94.21
```

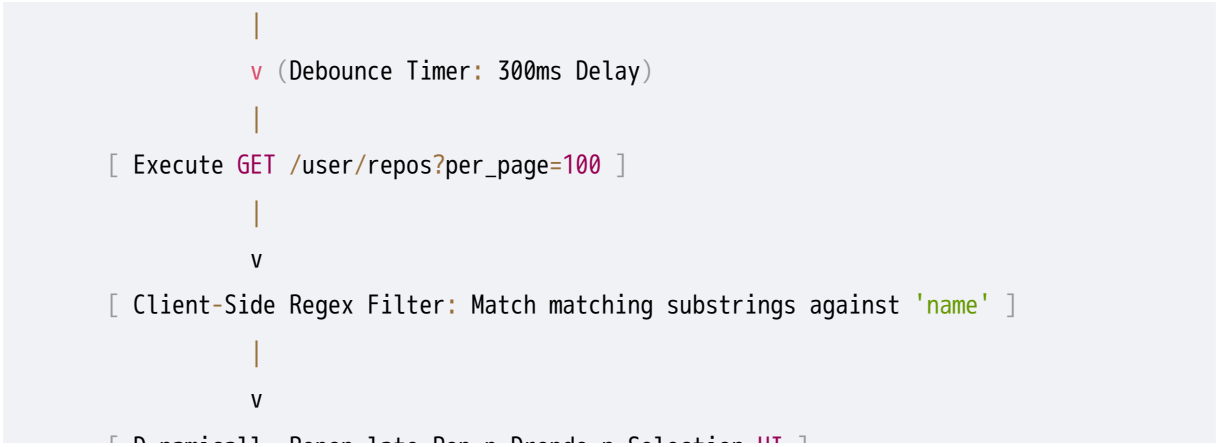
4. DYNAMIC GITHUB API INTEGRATION & REAL-TIME SEARCH FLOW

All interactions with GitHub utilize direct HTTP REST operations initialized from the Background Service Worker.

4.1 ONBOARDING & DYNAMIC REPOSITORY LOOKUP FLOW

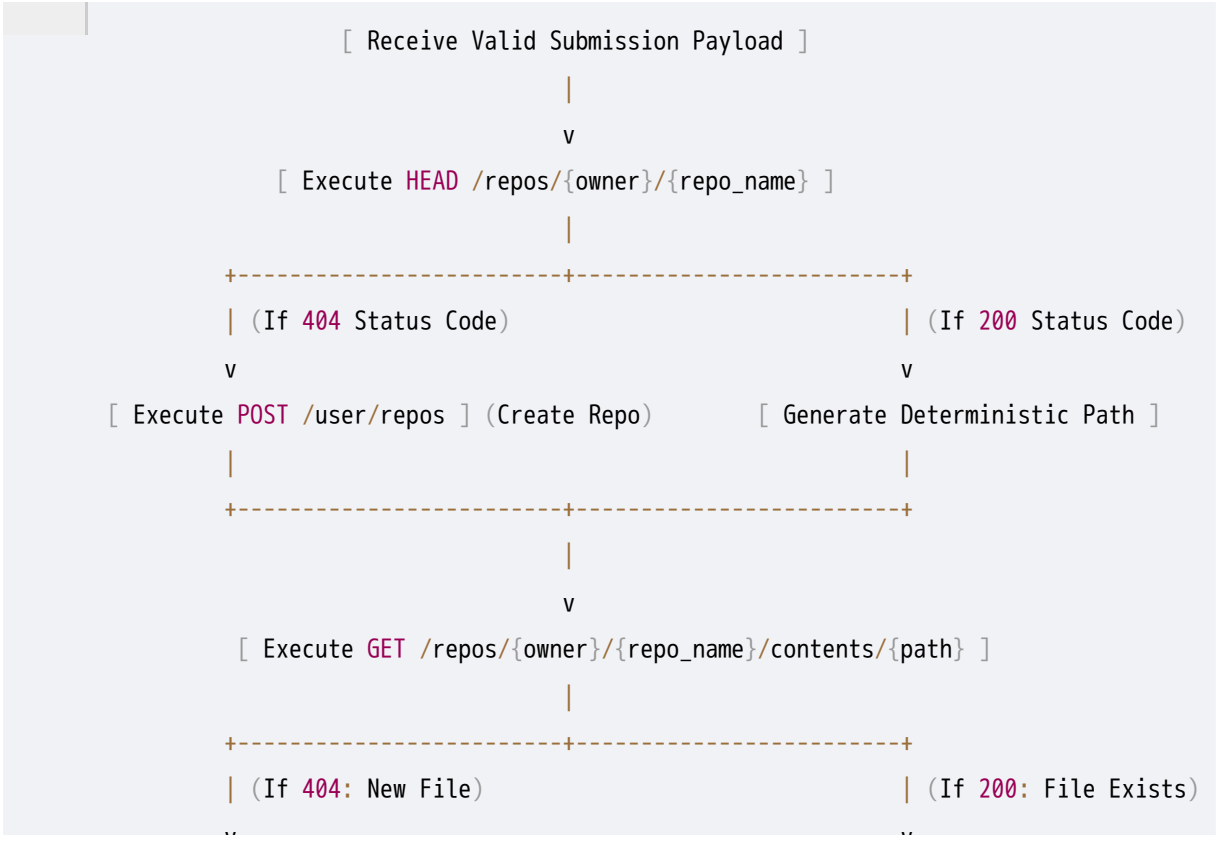
The extension popup UI provides a fluid onboarding experience leveraging debounced type-ahead search directly against GitHub endpoints.

```
[ User inputs GitHub PAT ]
          |
          v
[ Validate Token via GET /user ] ---> (If 401: Render "Invalid Token" Error)
          |
          v (If 200: Enable Repo Input Field)
          |
[ User types characters into Search Bar ]
```



4.2 BACKGROUND SYNCHRONIZATION ENGINE LOGIC

When a valid payload arrives from the Content Script, the Background Service Worker decrypts the PAT into memory and executes a linear validation and push sequence:



5. AUTOMATED "LAYMAN'S" README & REPOSITORY FILE STRUCTURE

To maintain a clean Git repository, files must be stored using a highly structured, predictable pathing hierarchy.

5.1 REPOSITORY FOLDER ARCHITECTURE

Every problem maps cleanly to its own isolated workspace path directory using the canonical ID and Slug schema:

```
📁 [Repository Root]/
│
└──── 📁 0001_two-sum/
    ├──── 📄 solution.py
    └──── 📄 README.md
```

5.2 ALGORITHMIC README GENERATION ENGINE

The extension features a deterministic markdown generation engine that parses the problem context scraped from the LeetCode state and constructs a highly readable documentation index (`README.md`).

Generation Template Architecture

The markdown content string is programmatically compiled using the exact formatting layout below:

Markdown

```
# 0001. Two Sum

## 📊 Properties
- **Difficulty:** Easy
- **Language:** Python 3
- **Runtime Performance:** 32 ms (Top 94.21%)
- **Memory Footprint:** 15.4 MB (Top 88.05%)
- **Problem Link:** [LeetCode - Two Sum](https://leetcode.com/problems/two-sum/)

---

## 📝 Problem Statement
Given an array of integers `nums` and an integer `target`, return indices of the two
numbers such that they add up to `target`.
You may assume that each input would have exactly one solution, and you may not use
the same element twice.

### Constraints
```

6. NON-FUNCTIONAL REQUIREMENTS & EDGE-CASE RESILIENCY

6.1 PERFORMANCE CONSTRAINTS

- **Asynchronous Non-Blocking Execution:** Every network interception, processing cycle, and file compilation execution block must process

asynchronously using Javascript `async/await` abstractions. Background workflows must yield execution prioritization to the browser UI threads, guaranteeing 0ms interface latency or tab lockups for the user.

- **Memory Footprint Ceiling:** Memory consumption during runtime operations inside the Background Service Worker must sit below a strict **20MB** upper bound. ArrayBuffers used for cryptography must be explicitly nullified or garbage collected immediately post-synchronization.

6.2 IDEMPOTENCY AND ANTI-SPAM CONTROL

To mitigate repository spamming when users repeatedly hit the "Submit" button on a pre-solved problem, the extension enforces a double-keyed local validation check.

- **Submission Deduplication Key:** The background script retains a local log of successfully pushed items inside `chrome.storage.local` indexed by a concatenated string: `${Problem_ID}_${Submission_ID}`.
- **Remote Content Comparison:** If a submission arrives matching a problem ID that was previously tracked but carries a *new* submission ID (e.g., optimization updates), the extension fetches the remote file's metadata (`GET /repos/{owner}/{repo}/contents/{path}`). It derives the SHA-1 hash of the newly compiled code block and compares it directly with the remote GitHub object asset blob SHA string (`sha` field). If the hashes are mathematically identical, the remote write step is bypassed, eliminating redundant commits.

6.3 RESILIENCY & OFFLINE QUEUE DATABASE LAYOUT

If a user solves problems during network connectivity blackouts, server side rate limits, or GitHub API outages (HTTP Status Codes `500` , `502` , `503` , or `429`), the extension switches into a fault-tolerant caching mode.

Storage Queue Schema (`chrome.storage.local`)

Failed sync tasks are written to an internal queue storage index:

JSON

```
{
  "offline_sync_queue": [
    {
      "retry_id": "uuid-v4-string-here",
      "retry_count": 0,
      "failed_at_timestamp": "2026-06-22T18:22:00Z",
```

```
            "payload": {
              "problem_id": "1",
              "file_path": "0001_two-sum/solution.py",
              "code_content": "base64_encoded_source",
              "readme_content": "base64_encoded_markdown"
            }
          }
        ]
      }
```

Reconnection & Retry Engine

1. The Background Service Worker listens to network status updates via the `navigator.onLine` event.
2. Additionally, a recurring background alarm (`chrome.alarms`) triggers every 30 minutes to wake the worker.
3. Upon wake or reconnection, the worker queries the `offline_sync_queue` .
4. If entries exist, it processes items one by one using an Exponential Backoff strategy.
5. If a retry fails, the entry's `retry_count` increments. Once an entry hits a max retry limit of 5 consecutive attempts, it is flagged as a permanent error, and a native browser notification alerts the user to manually re-verify their access configurations.